

Sample Project Documentation

Date	1 July 2026
Team ID	—
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	5 Marks

Project Description

Rising Waters is an end-to-end flood-risk prediction system built with Python, machine learning and Flask. Residents and disaster-management teams enter ten live weather and terrain readings — such as rainfall, river water level, humidity and drainage quality — into a simple web form and receive an instant Flood / No-Flood prediction, along with a probability score when the model supports it. The project follows the full applied-ML workflow: environment setup, dataset preparation, exploratory data analysis, preprocessing, multi-model training with automatic best-model selection, and deployment as a lightweight Flask web app.

Under the hood, four classifiers — Decision Tree, Random Forest, K-Nearest Neighbors and XGBoost — are trained and compared on the same preprocessed dataset. The best performer (XGBoost, ~75.3% test accuracy on the bundled dataset) is persisted along with its fitted StandardScaler and metadata, so the Flask app can load it once at startup and serve predictions with very low latency.

Scenarios

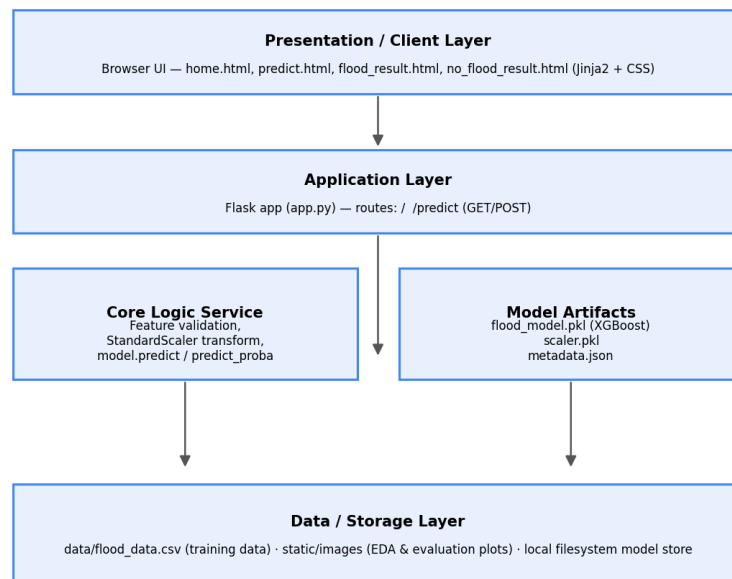
Scenario 1: A resident in a low-lying river-basin ward notices unusually heavy monsoon rain. They open the Rising Waters home page, click through to the prediction form, and enter today's rainfall, river level and humidity readings. The app returns a "Flood Chance" result with an estimated probability and safety guidance, prompting the resident to move belongings to higher ground.

Scenario 2: A local disaster-management officer is triaging several wards during a monsoon spell. For each ward, they quickly enter the latest weather-station and river-gauge readings into the form. Wards returning a high flood probability are flagged for early-warning resource allocation, while "No Flood Chance" wards are deprioritized.

Scenario 3: A team member re-runs the training pipeline after collecting a season's worth of new readings, regenerating `flood_model.pkl`, `scaler.pkl` and `metadata.json` so the live app reflects the most recent, most accurate model without any changes to `app.py`.

Technical Architecture

Solution Architecture — Rising Waters Flood Prediction System



Description: Rising Waters uses a three-layer architecture. The presentation layer is a set of Jinja2-rendered HTML pages (home, predict, flood_result, no_flood_result) styled with a shared stylesheet. The Flask application layer handles routing, form validation and orchestration. The core logic service applies the same fitted StandardScaler used during training, then runs the persisted best-performing classifier to produce a label and probability. Model artifacts (flood_model.pkl, scaler.pkl, metadata.json) and the source training dataset (data/flood_data.csv) make up the data layer. (Note: this project does not use a database — all data is a flat CSV file plus serialized model artifacts, so no ER diagram is required.)

Pre-requisites

- Flask Framework Knowledge: [Flask Documentation](#)
- scikit-learn & XGBoost Familiarity: [scikit-learn / XGBoost Documentation](#)
- HTML & CSS Skills: [W3Schools HTML/CSS Tutorials](#)
- Python Programming Proficiency: [Python Documentation](#)
- Version Control with Git: [Git Documentation](#)
- Development Environment Setup: [Anaconda / venv + pip install -r requirements.txt](#)

Project Workflow

Milestone 1: Environment Setup & Dataset

- Activity 1.1: Create and activate a Python 3.10 environment (conda or venv) and install dependencies from requirements.txt.
- Activity 1.2: Generate the working dataset with notebooks/0_generate_dataset.py, which simulates realistic flood-related weather and terrain data (including missing values and outliers) matching the structure of a real Kaggle flood dataset.

- Activity 1.3: Define the ten feature columns (ANNUAL_RAINFALL, SEASONAL_RAINFALL, CLOUD_VISIBILITY, HUMIDITY, TEMPERATURE, RIVER_WATER_LEVEL, WIND_SPEED, DRAINAGE_QUALITY, ELEVATION, HISTORICAL_FLOOD_OCCURRENCES) and the FLOOD_OCCURRED target.

Milestone 2: Exploratory Data Analysis

- Activity 2.1: Run notebooks/1_data_visualization.py to produce distribution/histogram plots, box plots for outlier detection, a correlation heatmap, feature-vs-target comparisons, and a multivariate pairplot, all saved to static/images/.
- Activity 2.2: Review printed descriptive statistics and missing-value counts to inform the preprocessing plan.

Milestone 3: Preprocessing & Model Building

- Activity 3.1: In notebooks/2_preprocessing_and_training.py, impute missing values with the column median and cap outliers using the IQR method.
- Activity 3.2: Split features/target, then create an 80/20 stratified train/test split, and fit a StandardScaler on the training features.
- Activity 3.3: Train Decision Tree, Random Forest, KNN and XGBoost (falling back to GradientBoostingClassifier if xgboost isn't installed) classifiers on the scaled data.
- Activity 3.4: Evaluate each model with confusion matrices, classification reports and accuracy, saving comparison charts (06_confusion_matrices.png, 07_accuracy_comparison.png).
- Activity 3.5: Automatically select the highest-accuracy model and persist it, the scaler, and metadata (best model name, accuracy, feature names) for the Flask app to consume.

Milestone 4: Flask Application Development

- Activity 4.1: Load the model, scaler and metadata once at app startup in app.py.
- Activity 4.2: Implement the home route (/) to display the active model name and its accuracy.
- Activity 4.3: Implement the GET /predict route to render the input form using a FEATURE_CONFIG dictionary of human-friendly labels, defaults and ranges.
- Activity 4.4: Implement the POST /predict route to read submitted values, scale them, run the model, and render either flood_result.html (with probability and safety guidance) or no_flood_result.html.
- Activity 4.5: Add error handling so invalid input redisplay the form with a clear message instead of crashing the app.

Model & scaler are loaded once at startup so every request is served without reload latency:

```
model = joblib.load(MODEL_PATH)
scaler = joblib.load(SCALER_PATH)
with open(METADATA_PATH) as f:
    metadata = json.load(f)

FEATURE_NAMES = metadata["feature_names"]
MODEL_NAME = metadata["best_model_name"]
MODEL_ACCURACY = round(metadata["accuracy"] * 100, 2)
```

The POST /predict route scales the submitted readings and returns the risk-specific page:

```
X = pd.DataFrame([input_values], columns=FEATURE_NAMES)
X_scaled = scaler.transform(X)
prediction = int(model.predict(X_scaled)[0])

probability = None
if hasattr(model, "predict_proba"):
    probability = round(float(model.predict_proba(X_scaled)[0][1]) * 100, 2)
```

```

if prediction == 1:
    return render_template("flood_result.html", probability=probability,
                           inputs=readable_inputs, model_name=MODEL_NAME)
else:
    return render_template("no_flood_result.html", probability=probability,
                           inputs=readable_inputs, model_name=MODEL_NAME)

```

Milestone 5: Frontend Development

- Activity 5.1: Build home.html, predict.html, flood_result.html and no_flood_result.html with Jinja2 templating.
- Activity 5.2: Style all pages with a shared static/css/style.css for a consistent, readable look.

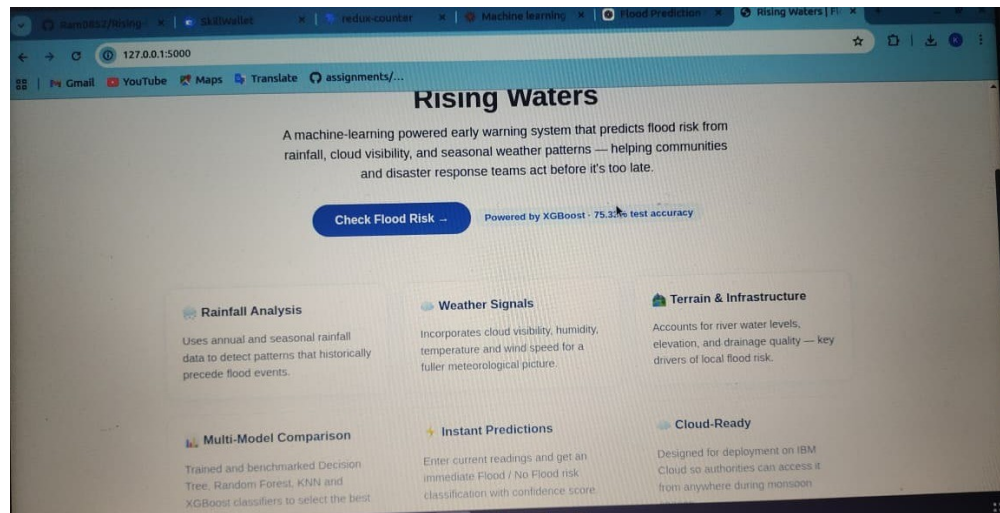
Milestone 6: Testing & Deployment

- Activity 6.1: Manually test the form with realistic and edge-case values across all ten features, and verify both result pages render correctly.
- Activity 6.2: Run the app locally with python app.py and confirm it is reachable at http://127.0.0.1:5000/.
- Activity 6.3 (optional): Prepare for IBM Cloud deployment by adding gunicorn to requirements.txt, reading the PORT environment variable, and pushing the repository to a Git host.

Exploring the Website's Web Pages

Home Page (/)

Description: The landing page introduces Rising Waters, explains its purpose, and displays the currently active machine-learning model along with its test accuracy, giving users confidence in the prediction before they proceed.



Prediction Form (/predict, GET)

Description: A form listing all ten weather/terrain features with friendly labels, units, sensible default values and min/max ranges, letting users quickly enter realistic readings even without domain expertise.

Enter Current Weather Readings

Fill in the values below to get an instant flood risk prediction.

Annual Rainfall (mm)	Seasonal Rainfall (mm)	Cloud Visibility (km)
2200	1100	6
Humidity (%)	Temperature (°C)	River Water Level (m)
70	28	5
Wind Speed (km/h)	Drainage Quality (1=Poor .. 5=Excellent)	Elevation (m above sea level)
12	3	120
Historical Flood Occurrences (last 10 yrs)		
1		

Predict Flood Risk

Flood Chance Result Page

Description: Shown when the model predicts a flood. Displays the estimated probability, the submitted input values, the model used, and plain-language safety guidance to help the user act quickly.

No Flood Chance Result Page

Description: Shown when the model predicts no flood. Displays the probability score and submitted inputs so users understand the basis for the assessment and can still stay alert if conditions change.

Below: the live app returning a Low Flood Risk (31.56% probability) result for the sample readings, followed by the submitted-readings summary and navigation options.

Low Flood Risk

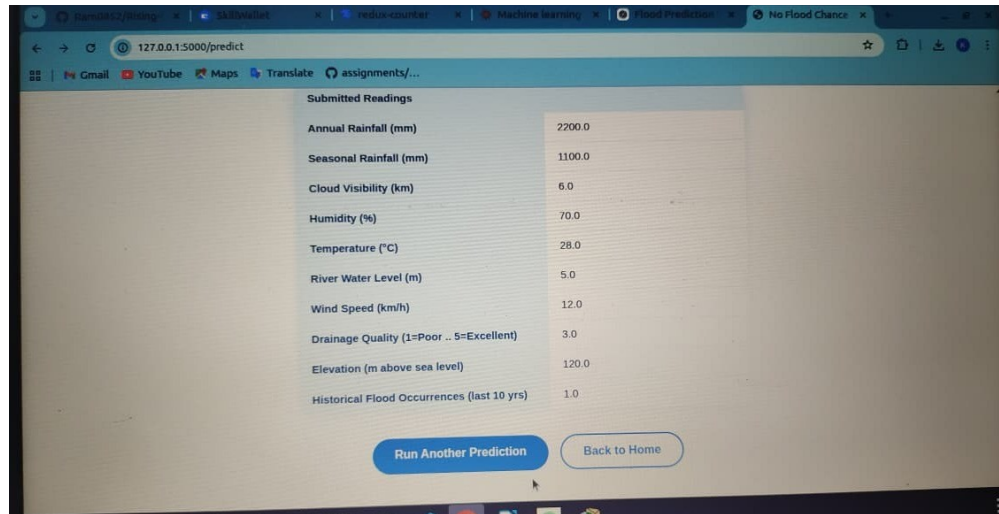
Predicted flood probability: **31.56%**

Model used: XGBoost

Recommendation: Current conditions indicate a low flood risk. Continue routine monitoring of rainfall and river levels, especially during monsoon season, and re-check predictions as new weather data becomes available.

Submitted Readings

Annual Rainfall (mm)	2200.0
Seasonal Rainfall (mm)	1100.0
Cloud Visibility (km)	6.0



Conclusion

Rising Waters demonstrates how a compact, well-structured machine-learning pipeline — spanning data simulation, EDA, preprocessing, multi-model comparison and a lightweight Flask front end — can turn raw weather and terrain readings into an actionable flood-risk assessment. By automatically selecting the best-performing classifier and exposing it through a simple form-based interface, the project makes data-driven flood awareness accessible to residents and disaster-management teams alike. Future enhancements could include integrating real, larger-scale datasets (e.g., from Kaggle or government hydrology sources), adding time-series forecasting for multi-day risk trends, and deploying the app publicly via IBM Cloud so it can be used in real disaster-response workflows.